

```

    string basename;
};
class derived : public base {
public:
    void print(ostream &os) { print(os); os << " " << i; }
private:
    int i;
};

```

在上述代码中存在问题吗？如果有，你该如何修改它？

练习 15.14：给定上一题中的类以及下面这些对象，说明在运行时调用哪个函数：

```

base bobj;          base *bp1 = &bobj;          base &br1 = bobj;
derived dobj;       base *bp2 = &dobj;          base &br2 = dobj;
(a) bobj.print();   (b) dobj.print();           (c) bp1->name();
(d) bp2->name();    (e) br1.print();            (f) br2.print();

```

15.4 抽象基类

假设我们希望扩展书店程序并令其支持几种不同的折扣策略。除了购买量超过一定数量享受折扣外，我们也可能提供另外一种策略，即购买量不超过某个限额时可以享受折扣，但是一旦超过限额就要按原价支付。或者折扣策略还可能是购买量超过一定数量后购买的全部书籍都享受折扣，否则全都不打折。

上面的每个策略都要求一个购买量的值和一个折扣值。我们可以定义一个新的名为 `Disc_quote` 的类来支持不同的折扣策略，其中 `Disc_quote` 负责保存购买量的值和折扣值。其他的表示某种特定策略的类（如 `Bulk_quote`）将分别继承自 `Disc_quote`，每个派生类通过定义自己的 `net_price` 函数来实现各自的折扣策略。

在定义 `Disc_quote` 类之前，首先要确定它的 `net_price` 函数完成什么工作。显然我们的 `Disc_quote` 类与任何特定的折扣策略都无关，因此 `Disc_quote` 类中的 `net_price` 函数是没有实际含义的。

我们可以在 `Disc_quote` 类中不定义新的 `net_price`，此时，`Disc_quote` 将继承 `Quote` 中的 `net_price` 函数。

然而，这样的设计可能导致用户编写出一些无意义的代码。用户可能会创建一个 `Disc_quote` 对象并为其提供购买量和折扣值，如果将该对象传给一个像 `print_total` 这样的函数，则程序将调用 `Quote` 版本的 `net_price`。显然，最终计算出的销售价格并没有考虑我们在创建对象时提供的折扣值，因此上述操作毫无意义。

纯虚函数

认真思考上面描述的情形我们可以发现，关键问题并不仅仅是不知道应该如何定义 `net_price`，而是我们根本就不希望用户创建一个 `Disc_quote` 对象。`Disc_quote` 类表示的是一本打折书籍的通用概念，而非某种具体的折扣策略。

我们可以将 `net_price` 定义成纯虚（pure virtual）函数从而令程序实现我们的设计意图，这样做可以清晰地告诉用户当前这个 `net_price` 函数是没有实际意义的。和普通的虚函数不一样，一个纯虚函数无须定义。我们通过在函数体的位置（即在声明语句

的分号之前) 书写=0 就可以将一个虚函数说明为纯虚函数。其中, =0 只能出现在类内部的虚函数声明语句处:

```
// 用于保存折扣值和购买量的类, 派生类使用这些数据可以实现不同的价格策略
class Disc_quote : public Quote {
public:
    Disc_quote() = default;
    Disc_quote(const std::string& book, double price,
               std::size_t qty, double disc):
        Quote(book, price),
        quantity(qty), discount(disc) { }
    double net_price(std::size_t) const = 0;
protected:
    std::size_t quantity = 0;           // 折扣适用的购买量
    double discount = 0.0;             // 表示折扣的小数值
};
```

和我们之前定义的 Bulk_quote 类一样, Disc_quote 也分别定义了一个默认构造函数和一个接受四个参数的构造函数。尽管我们不能直接定义这个类的对象, 但是 Disc_quote 的派生类构造函数将会使用 Disc_quote 的构造函数来构建各个派生类对象的 Disc_quote 部分。其中, 接受四个参数的构造函数将前两个参数传递给 Quote 的构造函数, 然后直接初始化自己的成员 discount 和 quantity。默认构造函数则对这些成员进行默认初始化。

值得注意的是, 我们也可以为纯虚函数提供定义, 不过函数体必须定义在类的外部。也就是说, 我们不能在类的内部为一个=0 的函数提供函数体。

610

含有纯虚函数的类是抽象基类

含有(或者未经覆盖直接继承)纯虚函数的类是**抽象基类**(abstract base class)。抽象基类负责定义接口, 而后续的其他类可以覆盖该接口。我们不能(直接)创建一个抽象基类的对象。因为 Disc_quote 将 net_price 定义成了纯虚函数, 所以我们不能定义 Disc_quote 的对象。我们可以定义 Disc_quote 的派生类的对象, 前提是这些类覆盖了 net_price 函数:

```
// Disc_quote 声明了纯虚函数, 而 Bulk_quote 将覆盖该函数
Disc_quote discounted;           // 错误: 不能定义 Disc_quote 的对象
Bulk_quote bulk;                 // 正确: Bulk_quote 中没有纯虚函数
```

Disc_quote 的派生类必须给出自己的 net_price 定义, 否则它们仍将是抽象基类。



我们不能创建抽象基类的对象。

派生类构造函数只初始化它的直接基类

接下来可以重新实现 Bulk_quote 了, 这一次我们让它继承 Disc_quote 而非直接继承 Quote:

```
// 当同一书籍的销售量超过某个值时启用折扣
// 折扣的值是一个小于1的正的小数值, 以此来降低正常销售价格
class Bulk_quote : public Disc_quote {
public:
    Bulk_quote() = default;
```

```

Bulk_quote(const std::string& book, double price,
           std::size_t qty, double disc):
    Disc_quote(book, price, qty, disc) { }
// 覆盖基类中的函数版本以实现一种新的折扣策略
double net_price(std::size_t) const override;
};

```

这个版本的 Bulk_quote 的直接基类是 Disc_quote，间接基类是 Quote。每个 Bulk_quote 对象包含三个子对象：一个（空的）Bulk_quote 部分、一个 Disc_quote 子对象和一个 Quote 子对象。

如前所述，每个类各自控制其对象的初始化过程。因此，即使 Bulk_quote 没有自己的数据成员，它也仍然需要像原来一样提供一个接受四个参数的构造函数。该构造函数将它的实参传递给 Disc_quote 的构造函数，随后 Disc_quote 的构造函数继续调用 Quote 的构造函数。Quote 的构造函数首先初始化 bulk 的 bookNo 和 price 成员，当 Quote 的构造函数结束后，开始运行 Disc_quote 的构造函数并初始化 quantity 和 discount 成员，最后运行 Bulk_quote 的构造函数，该函数无须执行实际的初始化或其他工作。

611

关键概念：重构

在 Quote 的继承体系中增加 Disc_quote 类是重构（refactoring）的一个典型示例。重构负责重新设计类的体系以便将操作和/或数据从一个类移动到另一个类中。对于面向对象的应用程序来说，重构是一种很普遍的现象。

值得注意的是，即使我们改变了整个继承体系，那些使用了 Bulk_quote 或 Quote 的代码也无须进行任何改动。不过一旦类被重构（或以其他方式被改变），就意味着我们必须重新编译含有这些类的代码了。

15.4 节练习

练习 15.15：定义你自己的 Disc_quote 和 Bulk_quote。

练习 15.16：改写你在 15.2.2 节（第 533 页）练习中编写的数量受限的折扣策略，令其继承 Disc_quote。

练习 15.17：尝试定义一个 Disc_quote 的对象，看看编译器给出的错误信息是什么？



15.5 访问控制与继承

每个类分别控制自己的成员初始化过程（参见 15.2.2 节，第 531 页），与之类似，每个类还分别控制着其成员对于派生类来说是否可访问（accessible）。

受保护的成员

如前所述，一个类使用 protected 关键字来声明那些它希望与派生类分享但是不想被其他公共访问使用的成员。protected 说明符可以看做是 public 和 private 中和后的产物：

- 和私有成员类似，受保护的成员对于类的用户来说是不可访问的。